

TicketSphere

AI ticket intelligence for application maintenance

A multi-agent pipeline that reads every Jira ticket, retrieves what the organisation already knows about failures of that shape, and produces a cited, scored, auditable decision — never silently, never without a reason a reviewer can point at.

Application Maintenance / ITSM • Ticketing system of record: Jira Cloud

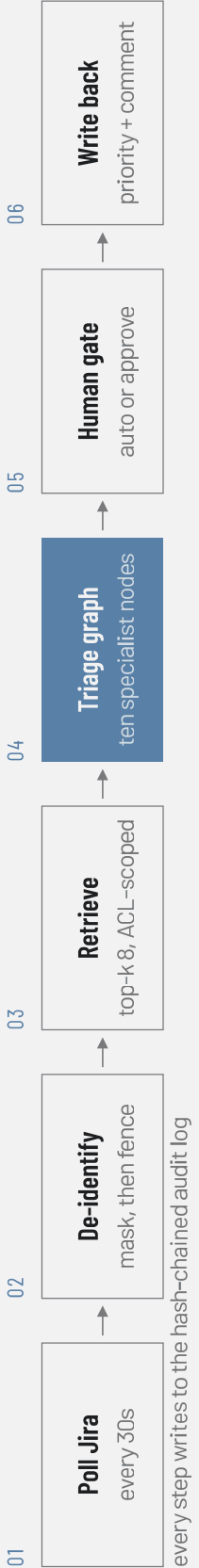
01 · THE PROBLEM

Triage today is slow, inconsistent, and undocumented

- + Four platform teams — Ops, Azure, AWS, GCP — take outages, slow queries, access requests and deployment failures, all day
- + A human reads each ticket, guesses how bad it is, guesses who owns it, and moves it
- + The guess is inconsistent between people, and inconsistent for the same person on a Friday evening
- + Nobody can reconstruct why a ticket was called a P1 last March

03 · SYSTEM OVERVIEW

One pipeline, board to board



The rest of the pipeline is byte-for-byte identical whether tickets come from the live Jira adapter or the synthetic seed — the offline fallback if the network fails.

The knowledge base is already indexed

CORPUS	DOC_TYPE	WHAT IT IS
Historical tickets	ticket_history	Resolved incidents with their real resolution times — the precedent library
Runbooks	runbook	documents of procedures, troubleshooting guide and expected outcomes for several incidents
Service catalogue	service_catalog	Which team owns which service
SLA policy	sla_policy	Priority → response and resolution minutes
Escalation Rules	escalation_matrix	When a human must be involved (restricted sensitivity)

Every document is chunked, embedded, and stored with governance metadata — allowed_roles, sensitivity, team, doc_type — so access control is applied inside the search query, not filtered afterwards.

04 · ACT 0 — BEFORE ANYONE LOGS IN

Ingestion and the background threads

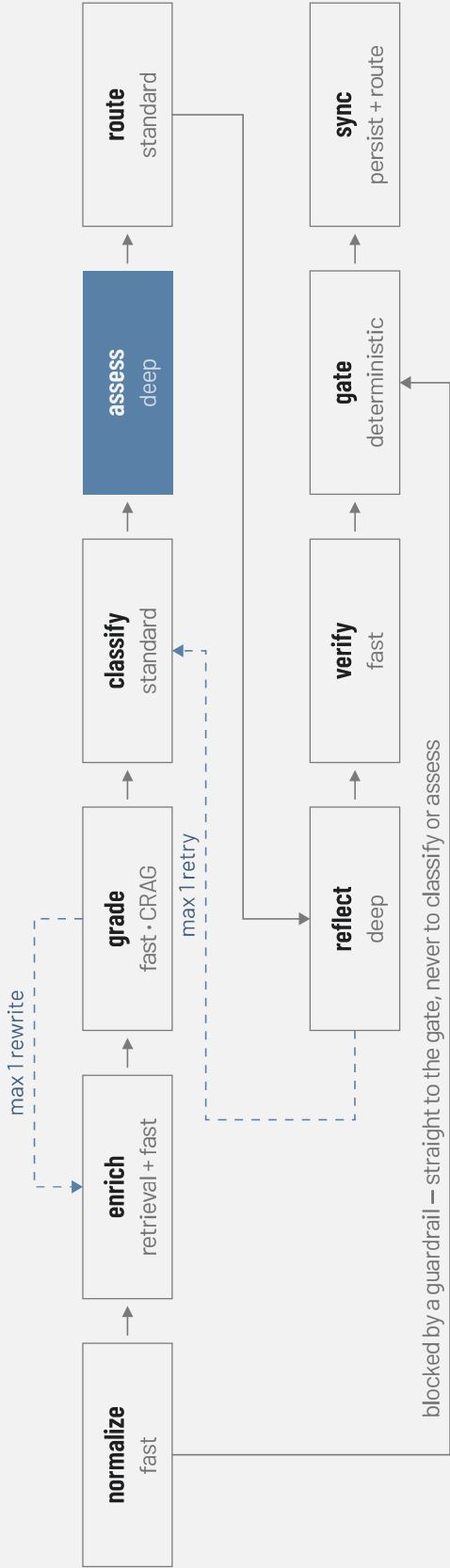
- + Boot launches two daemon threads: the ticket poller, running a sync cycle every 30 seconds
- + ...and the SLA monitor, which every 15 minutes emails managers about any open ticket that has burned 70% of its resolution target
- + The product is already ingesting before anyone clicks anything — Refresh only shows it happening on demand

•

Managers add documents at any time on the Knowledge Base screen — PDF upload or paste

07 · ACT 3 — INSIDE ONE TICKET

The ten-node triage graph



07 · ACT 3 — INSIDE ONE TICKET

Nodes 1–5: understand, then decide

NODE	WHAT IT DOES
1 · normalize	Clean and structure ticket
2 · enrich	Retrieve knowledge base, runbook, catalogue and SLA policy
3 · grade	Corrective RAG — check retrieval quality
4 · classify	determining categories - azure/aws/gcp/ops
5 · assess	setting Priority against the SLA policy and priority rulebook

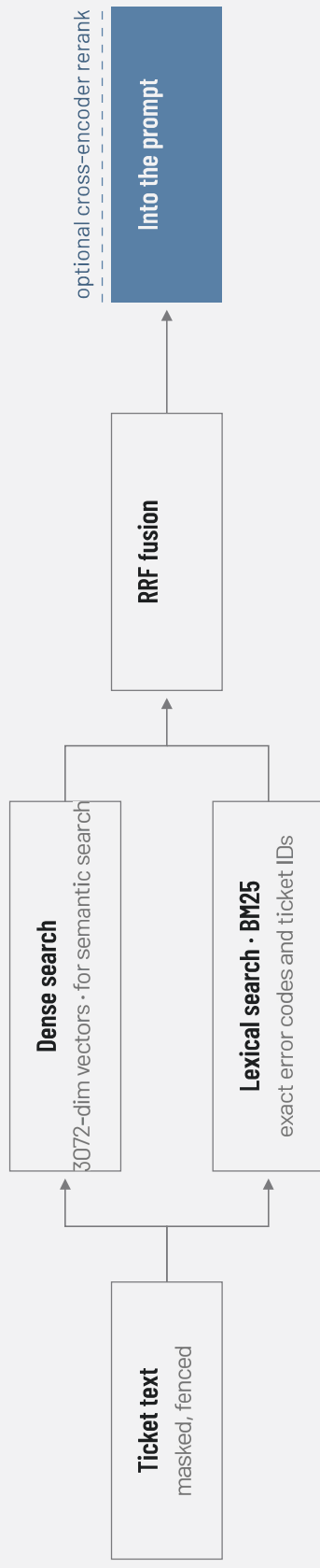
07 · ACT 3 — INSIDE ONE TICKET

Nodes 6-10: check, then act

NODE	WHAT IT DOES
6 · route	determining Owning team from the service catalogue
7 · reflect	Self-critique the decision — it may only lower confidence, never raise it
8 · verify	Output guardrails, policy, PII leak
9 · gate	Human-in-the-loop decision — does this need a senior approval?
10 · sync	Assemble the decision, decide auto-route or send to approval queue

08 · RETRIEVAL

The RAG pipeline, end to end



Nothing acts without a person when it matters

OUTCOME	TRIGGER	WHAT HAPPENS NEXT
Routed	Confidence ≥ 0.50 , priority below Highest, no flags	Auto-approved — written back to Jira immediately
Awaiting approval	Highest priority, thin evidence, a coin-flip band, or a duplicate	Lands in the manager's approval queue

15 · ACT 5 — TWO-WAY SYNC

Every approved decision writes back to Jira

- + **Update Priority in Jira**
- + **Add team label** — AWS, AZR, GCP or OPS
- + **A comment and a transition** — the full decision in Jira's own format, then In Progress or Done; an unknown transition is skipped with a warning, never a crash

TicketSphere auto-approved: **High · aws** · confidence **87%**. Payment callback timeouts match INC00000042's connection-pool exhaustion [C2]; SLA resolve target 24h per the policy matrix [C4].

Recommended resolution: check the Lambda's reserved concurrency against the RDS Proxy max connections before increasing the timeout.

16 · STATUS

Fully built and running today

- + The ten-node triage graph, both retry loops, and the escalation ladder
- + Hybrid retrieval — dense, BM25 and RRF — with CRAG grading and query rewrite
- + Chunk-level ACL applied inside the vector query, and its SQL equivalent
- + Input and output guardrails, PII masking, and the hash-chained audit log
- + Live Jira poll and write-back — priority, labels, comment, transition — watermarked and idempotent
- + Confidence as min(gates), with named gates and a recorded limiting gate
- + The full console: queue, history, triage, control tower, knowledge base, evals, usage, chat